

IIP Kiosk Project - Change Summary and Code Explanation

Updated on 30 June 2026. This report explains the completed feedback, pulse, tree, email, passport, animation, and admin parameter changes for the final year project. It also records the bugs fixed during the review and what each relevant code area now does.

1. Overall System Idea

The kiosk collects visitor feedback, optional pledge information, optional photo data, and email details. After submission, the visitor earns a badge and a leaf. The same submission can appear in three visitor-facing experiences: the feedback thank-you animation, the public pulse screen, and the digital tree screen. Admins can control text, feature toggles, validation rules, page styling, backgrounds, badge leaf colours, and shared leaf size through the parameter adjustment page.

The central design choice is to keep editable values in one parameter configuration object. The backend serves that object through `/api/parameters`, while the admin page saves it through `/api/admin/parameters`. Visitor screens read the same configuration, so the system behaves consistently across feedback, pulse, and tree.

2. Feedback Screen

- Quick Consent was removed from the visitor flow. Feedback is treated as long-term by default, so the visitor no longer chooses temporary versus long-term retention before entering the form.
- The confirm submission page no longer presents Data Retention and Primary Consent in the visible summary list. This reduces repeated consent wording and matches the long-term-only requirement.
- The thank-you social buttons are controlled by the admin feature flag `socialSharingEnabled`. When the flag is false, the visitor does not see the social sharing buttons.
- The thank-you and badge emails were merged into one response email. The merged email can include the thank-you message, badge result, visitor photo/passport content, and badge highlights instead of sending two separate emails.
- The visitor passport is mapped by email. Each time a visitor completes feedback with the same email address, the passport can show their own history and collected badges, giving repeat visitors a real passport-style record.
- The feedback thank-you page now plays a flying leaf animation. The selected badge controls the leaf colour and target area, then the leaf visually joins the mini tree preview.

Feedback Code Details

- `frontend/feedback/feedback.js` loads `kioskParameters` from `/api/parameters`. `applyParameterOverrides()` applies admin-controlled text, feature flags, validation rules, layout settings, feedback background CSS, card opacity, accent colour, and celebration leaf scale.
- `getConfiguredBadgeLeafStyles()` builds the final badge colour map. It starts with safe fallback colours, then overlays admin values from `badgeLeafStyles.colors`. This means admin-selected badge colours override hardcoded defaults.
- `getCelebrationBadgeColor()` uses the configured badge map so the colour in the thank-you animation matches the badge leaf colour used on the tree and pulse screens.
- `setupCelebrationMoment()` sets CSS variables such as `--celebration-badge-color`, `--celebration-leaf-scale`, `--celebration-leaf-image`, and the flight path variables. The CSS animation reads those variables to fly the leaf into the tree.
- `frontend/feedback/feedback.css` defines `--feedback-card-opacity`, `--feedback-accent-color`,

- --feedback-accent-dark, and --celebration-leaf-scale. Cards, buttons, pledge coach highlights, and the flying leaf animation use these variables.
- backend/feedbackRoutes.js handles feedback submission, stores metadata, saves the badge result, and supports the visitor passport endpoint. The passport endpoint groups records by email so the right person sees their own repeated-feedback history.
- backend/emailService.js builds the merged email response. It uses visitor, badge, photo, and passport information so the email is more useful and avoids duplicate separate responses.

3. Admin Parameter Interface

- The admin parameter page now includes Feedback Page Styling controls in the Backgrounds and Digital Overlays tab. Admins can set Feedback Background CSS, upload a background image, adjust Feedback Card Opacity, and choose the Feedback Accent Colour.
- The Digital Tree Visualization tab now includes Badge Leaf Styling controls. Admins can change the shared leaf size and choose a colour for each badge: Feedback Contributor, Climate Champion, Renewable Innovator, Sustainable Living Advocate, Ocean Guardian, Governance Guardian, and Social Champion.
- The shared leaf size is saved to both treeParameters.leafDisplayScale and badgeLeafStyles.leafScale. This keeps normal green leaves and badge-coloured leaves synchronized.

Admin Code Details

- backend/parametersConfigStore.js adds feedbackPageStyle and badgeLeafStyles defaults. feedbackPageStyle stores backgroundCss, cardOpacity, and accentColor. badgeLeafStyles stores leafScale and a colours object keyed by badge key.
- mergeWithDefaults() deep-merges badgeLeafStyles.colors. This protects older configuration files from missing new badge colour keys after an update.
- backend/adminRoutes.js accepts feedbackPageStyle and badgeLeafStyles in PUT /api/admin/parameters. It validates that they are objects and merges the new values into the existing config.
- backend/adminRoutes.js also updates the background upload route. Uploaded feedback backgrounds are written to both visualAssets.feedbackBackground and feedbackPageStyle.backgroundCss so the new field does not block uploaded backgrounds from appearing.
- frontend/admin/admin.html adds the visible controls: card opacity range slider, accent colour picker, shared leaf size range slider, and per-badge colour pickers.
- frontend/admin/admin.js populates the controls from config, binds live previews, collects form values, and saves them back in the payload. collectParameterForm() is the function that packages the admin selections into the JSON sent to the backend.
- frontend/admin/admin.css styles the new admin controls. The badge preview cards show a small leaf-shaped preview and update when the colour or size changes.

4. Pulse Screen

- The pulse screen is public. Public visitors can view it without admin login, which allows it to run beside the tree on the large display.
- The pulse layout is designed for a 7m by 2m display and should not require scrolling. It uses a dense public dashboard layout for pledge metrics, recent pledges, badge breakdown, and the tree preview.
- Pulse leaves are placed near branch anchor points and include visitor names inside the leaf, similar to the /tree screen.
- Pulse now receives badgeKey, badgeColor, and leafScale from the backend summary API. That allows pulse leaves to use the same admin-controlled badge colour and shared size as the main tree.

Pulse Code Details

- backend/pulseRoutes.js exposes GET /api/pulse/summary. It aggregates pledges today, pledges this month, total pledges, campaign progress, newest pledges, top badge earners, badge breakdown, and tree visitors.
- topicToBadge() converts pledge topics into human-readable badge names. topicToBadgeKey() converts the same topics into stable keys used by CSS and configuration. Skipped or empty pledges become feedback-completer.
- The pulse summary reads badgeLeafStyles from parametersConfigStore and attaches badgeColor and leafScale to each tree visitor item.
- frontend/pulse/pulse.js renders metrics, newest pledges, badge rows, and the tree. renderTree() now uses visitor.badgeColor and visitor.leafScale instead of only fixed leaf images.
- frontend/pulse/pulse.css uses .pulse-leaf-tinted to mask the leaf shape and fill it with the configured colour. Visitor names remain displayed inside the leaf.

5. Tree Screen

- The feedback box below the leaf was removed from the tree leaf display. This makes the screen cleaner and focuses attention on the tree, badge, visitor name, and leaf detail.
- Leaves have an appearing animation when they enter the tree. After the configured leaf threshold is reached, leaves can drop/fall away using the leaf fall animation.
- Badge leaves can turn green at the configured daily reset time, while VIP leaves remain gold. This supports the public tree growing cycle.
- Badge leaf colours are now exact admin-selected colours, not approximations from hue-rotate filters. This makes the admin colour picker trustworthy.

Tree Code Details

- frontend/tree/tree.js fetchParameterConfig() reads treeParameters, visualAssets, campaignSettings, and badgeLeafStyles from /api/parameters. It applies canopy size, refresh interval, opacity, animation timings, green reset time, tree stage, background, leaf override image, leaf display scale, and badge colour map.
- createLeaf() creates each visitor leaf as a div. It decides if the visitor is VIP, whether the leaf should be greenified, which badge profile should be used, what image side to use, what size the leaf should be, and where it should appear in the tree mask.
- getBadgeLeafColor() picks the configured colour for the visitor badge key, falling back to the backend badge colour or default green.
- frontend/tree/tree.css keeps the existing leaf image structure but adds .leaf-tinted. The pseudo-element uses the leaf PNG as a mask and fills it with --leaf-badge-color. Leaf names remain on top because the mask is applied to ::before, not the text layer.
- backend/treeRoutes.js supplies tree visitor records with badgeKey, badgeName, badgeColor, pledgeTopic, name, visit count, and created date. The frontend uses these fields to render the correct public tree leaf.

6. Bugs Fixed During Review

- Background upload priority bug: the new feedbackPageStyle.backgroundCss field could override a newly uploaded background that was stored only in visualAssets.feedbackBackground. The upload route now writes both fields.
- Palette fallback bug: admin badge leaf colour controls originally risked falling back to the email badge palette. A separate DEFAULT_BADGE_LEAF_COLORS map now keeps visual leaf colours independent from email template colours.
- Pulse badge mismatch: pulse previously had only a badge label, so it could not reliably map to the same leaf colour used by the tree. The backend now sends badgeKey, badgeColor, and leafScale.

- Badge colour accuracy: tree leaves previously depended on hue-rotate CSS classes. The new mask approach displays the exact selected hex colour.
- Shared size drift: coloured leaves and green leaves could diverge if controlled separately. The admin save logic now writes one shared value into both relevant config fields.

7. Data and Rendering Flow

Admin UI -> collectParameterForm() -> PUT /api/admin/parameters -> parametersConfigStore.writeParametersConfig()
 /api/parameters -> feedback.js, pulse.js, tree.js -> CSS variables and render logic
 Feedback submit -> backend feedback routes -> badge metadata -> email service + visitor passport + public tree/pulse data

This flow keeps admin decisions in one configuration layer and makes each public screen read from that layer. When a badge colour changes, the same colour is used in the thank-you animation, pulse leaf, and tree leaf after the visitor screens reload or refresh.

8. Important Configuration Fields

- feedbackPageStyle.backgroundCss: CSS background value for the feedback page. Example: url("/assets/backgrounds/file.jpg") center / cover no-repeat fixed.
- feedbackPageStyle.cardOpacity: number between 0.55 and 1.00. Lower values show more of the background image through the cards.
- feedbackPageStyle.accentColor: hex colour used by feedback buttons and accents. The frontend derives a darker shade for gradients.
- badgeLeafStyles.leafScale: shared scale for badge leaves. It is kept in sync with treeParameters.leafDisplayScale.
- badgeLeafStyles.colors: object keyed by badge key. Each value is a hex colour used for the matching leaf.
- treeParameters.leafDisplayScale: scale used by the main digital tree when calculating leaf width and height.
- featureFlags.socialSharingEnabled: controls whether the thank-you page social media buttons are visible.

9. How To Use In Admin

- Open Admin -> Parameter Adjustment -> Backgrounds and Digital Overlays to change the feedback page background, opacity, and accent colour.
- Use Upload Feedback Background to add a PNG, JPG, or WebP. The upload route saves it immediately as the active feedback background.
- Open Admin -> Parameter Adjustment -> Digital Tree Visualization Parameters to change the shared leaf size and badge leaf colours.
- Save parameters after changing sliders or colour pickers. Public screens should refresh or reload to use the latest configuration.

10. Verification Completed

- node --check backend/adminRoutes.js passed.
- node --check backend/parametersConfigStore.js passed.
- node --check backend/pulseRoutes.js passed.
- node --check frontend/admin/admin.js passed.
- node --check frontend/feedback/feedback.js passed.
- node --check frontend/pulse/pulse.js passed.
- node --check frontend/tree/tree.js passed.
- git diff --check passed with only existing LF/CRLF line-ending warnings.

11. Notes For Final Year Project Explanation

The important engineering idea is separation of control and rendering. Admin pages do not directly modify public page HTML. Instead, they save structured configuration. Public pages fetch that configuration and translate it into CSS variables, render data, and animations. This makes the project easier to explain, test, and extend because the same parameter object controls the same behaviour everywhere.

The badge passport feature is a user-history feature. It uses the visitor email as the identity key, so the system can show repeated feedback achievements to the same visitor. The badge colour feature is a visual-configuration feature. It uses stable badge keys so display code does not depend on changing human-readable labels.

The leaf animation and mask work is primarily frontend rendering. JavaScript chooses the badge, colour, image, size, and flight path. CSS performs the visual animation. This keeps animation timing and shapes in CSS while keeping visitor-specific data in JavaScript.